

ENGINEERING · INTERNAL PITCH · 2026

Engineering System for AI-Assisted Development.

Strategy → Spec → Execution → Learning Loop

AUDIENCE

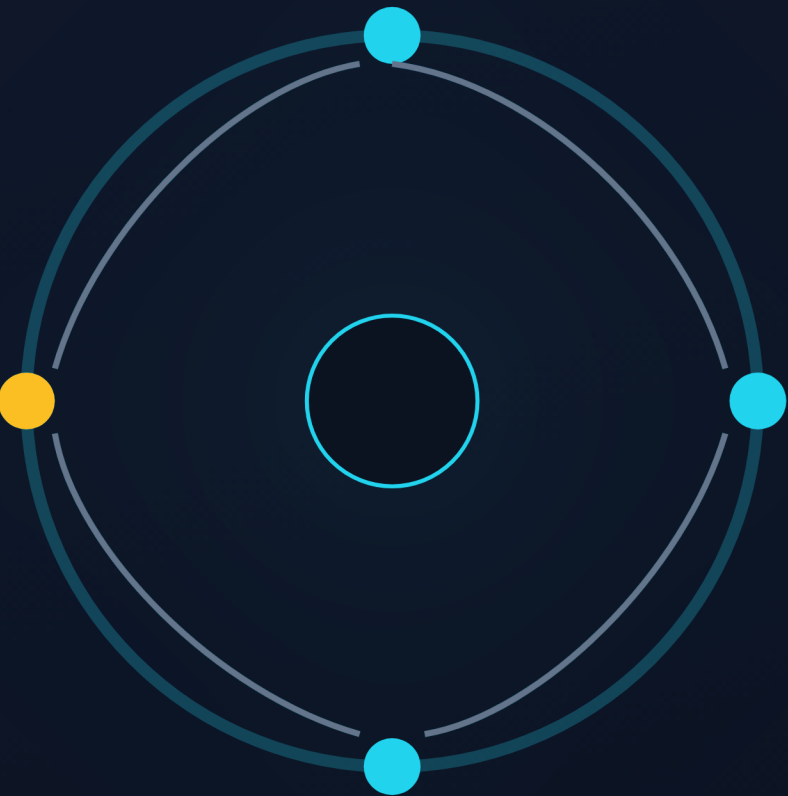
Engineering & Leadership

READ TIME

~15 minutes

STATUS

Proposal



PROBLEM

AI coding is fast — but unstructured.

Requirements live in chat.

Intent is captured in transient threads, not in the repo.
A week later, no one can answer why a feature was built the way it was.

PRs lose context.

The reasoning happens upstream of the diff.
Reviewers see code without the decisions that produced it.

No shared engineering memory.

Two engineers solve the same problem twice. Patterns aren't captured; pitfalls aren't named.

Inconsistent review quality.

Every reviewer applies a different lens. Standards drift across features and across teams.

DIAGNOSIS

We don't lack AI capability. We lack engineering structure.

WHAT'S MISSING

Four gaps the AI can't fill on its own.

- No strategy layer — direction is implicit.
- No spec discipline — intent stays in chat.
- No execution consistency — every engineer invents a workflow.
- No feedback loop — learnings evaporate after each ship.

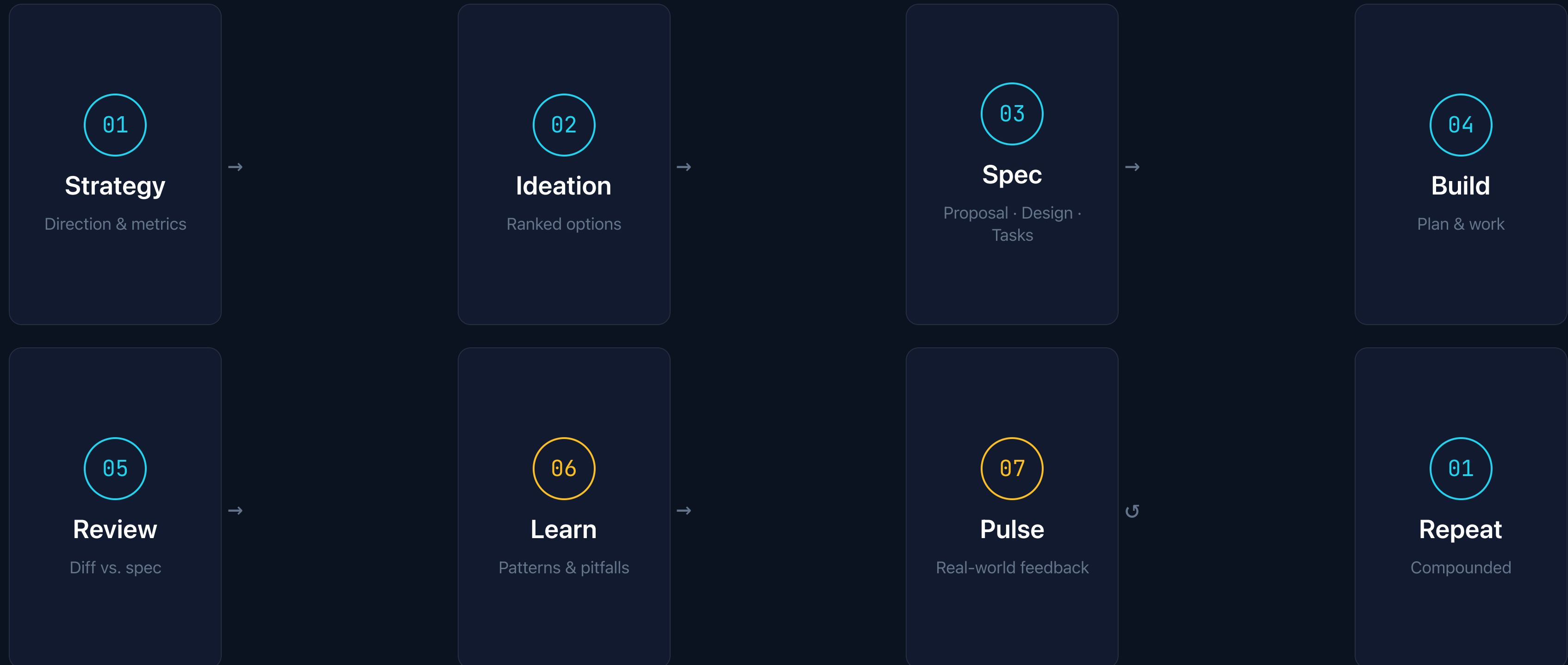
WHAT WE ADD

Structure around the model — not a better model.

- + A named strategy step that produces an artifact.
- + A spec layer with proposal, design, tasks.
- + A standard lifecycle every engineer follows.
- + A learning loop that compounds across features.

PROPOSAL

A closed-loop engineering system.



Two tools. One lifecycle. Complementary, not overlapping.

SPEC LAYER

OpenSpec

@fission-ai/openspec

Turns a feature idea into structured artifacts before any code is written: `proposal.md`, `design.md`, `tasks.md`.

Answers: what to build.

+

LIFECYCLE LAYER

Compound Engineering

EveryInc/compound-engineering-plugin

Drives execution, review, and learning through Claude Code slash commands — and feeds insights back into strategy.

Answers: how to build & how to learn.

We're not the only ones noticing this.

A whole category — Spec-Driven Development — has emerged in the last 18 months. Eight frameworks now compete to put structure around AI-assisted engineering. We evaluated them all.

SPEC LAYER

What to build

Turn intent into proposal · design · tasks before code is written.

LIFECYCLE LAYER

How to build

Drive plan → execute → review → learn through named, repeatable steps.

SKILLS LAYER

Reusable capabilities

Procedural know-how installed once, reused across every agent and feature.

EVIDENCE

AI velocity is also AI risk — at scale.

9.8—
42.1

% of LLM-generated
code is vulnerable.

AI code generation works at scale, and so do its vulnerabilities. Surviving AI-introduced issues in production repositories had topped 110,000 by February 2026. Spec-Driven Development embeds executable specifications as active validation gates against exactly these failures.

— YAN ET AL., 2025 • AUGMENT CODE, 2026

EIGHT FRAMEWORKS · EVALUATED END-TO-END

The field, at a glance.

SPEC · MICROSOFT-BACKED

Spec-Kit

github/spec-kit

GitHub's opinionated SDD CLI. Strong on executable specs as validation gates; tight GitHub workflow integration.

CONSIDERED

SPEC · OUR PICK

OpenSpec

@fission-ai/openspec

Clean three-artifact model — proposal · design · tasks. Lightweight, IDE-agnostic, easy to bend to our process.

SELECTED

METHOD · MULTI-AGENT

BMAD Method

bmad-method.org

Agentic teams (analyst, PM, architect, dev). Heavy framework — more ceremony than a 6-person team needs day one.

CONSIDERED

LIFECYCLE · OUR PICK

Compound Engineering

EveryInc

Lifecycle commands plus an explicit learning loop. Pairs cleanly with OpenSpec; covers what OpenSpec doesn't.

SELECTED

SPEC · COMMERCIAL

Tessl

tessl.io

Commercial SDD platform. Mature tooling, but vendor lock-in and pricing are friction for a starter project.

CONSIDERED

SPEC · IDE-NATIVE

Kiro

kiro.dev

Spec-driven IDE from AWS. Powerful, but couples the workflow to a specific editor we don't standardize on.

CONSIDERED

SKILLS · COMPLEMENT

Superpowers

obra/superpowers

Agentic skills framework. Procedural know-how installed once, reused across agents — complements our spec layer.

LAYER 3

SKILLS · COMPLEMENT

Agent Skills · skills.sh

addyosmani/agent-skills

Production-grade skill packs for AI coding agents. One-command install. Stackable on top of our chosen combo.

LAYER 3

RECOMMENDATION FOR THE FACTORY PROJECT

OpenSpec + Compound Engineering.

One framework for *what to build*, one for *how to build & learn*. Skills sit on top as a third, optional layer. Together they cover the full lifecycle without overlap.

SPEC LAYER

OpenSpec

Wins because the artifact model — proposal, design, tasks — is the cleanest in the field, and it's IDE-agnostic.

- ✓ Lightweight: one CLI, no platform lock-in
- ✓ Three artifacts map to how engineers already think
- ✓ Easy to template, easy to enforce in CI
- ✓ Beats Spec-Kit on flexibility, Tessel on cost, Kiro on portability

LIFECYCLE LAYER

Compound Engineering

Wins because it's the only framework with an explicit learning loop — the part that turns shipped features into team knowledge.

- ✓ Named commands for every phase — hard to skip
- ✓ `/ce-compound` + `/ce-product-pulse` close the loop
- ✓ Pairs cleanly with OpenSpec — no overlap
- ✓ Lighter than BMAD, more mature than skills-only stacks

MATCH THE PROCESS TO THE TASK

Three tiers. One rule: when in doubt, go up.

TIER 01

Hotfix

Bug, typo, or config tweak with obvious scope.

< 2 HOURS

```
/ce-debug →  
/ce-code-review
```

TIER 02

Standard Feature

Clear requirements that fit inside a single sprint.

< 1 WEEK

```
/opsx:propose →  
/ce-plan → /ce-work →  
/ce-code-review → /ce-compound
```

TIER 03

Major Feature

Architectural change, cross-team work, or unclear scope.

> 1 WEEK

```
/ce-strategy → /ce-ideate →  
/opsx:propose → /opsx:apply →  
/ce-plan → /ce-work →  
/ce-code-review → /ce-compound →  
/ce-product-pulse
```

THE FULL LIFECYCLE, IN COMMANDS

Eight named steps. Each one validates the last.

01

`/ce-strategy`

Direction, users,
metrics,
constraints.

02

`/ce-ideate`

Explore & rank
solution options.

03

`/opsx:propose`

Generate
proposal, design,
tasks.

04

`/ce-plan`

Sequence work,
resolve
dependencies.

05

`/ce-work`

Execute, task by
task, with the
human in the loop.

06

`/ce-code-review`

Review the diff
against the spec.

07

`/ce-compound`

Capture patterns,
pitfalls, decisions.

08

`/ce-product-pulse`

Real-world
feedback feeds
the next strategy
cycle.

Why it works.

01 · SEPARATION

Thinking before execution.

The hard product thinking happens before the AI writes code. The model receives a well-formed problem, not a vague one.

02 · DISCIPLINE

Clarity before code.

A bad spec is cheap to fix. Code that implements a bad spec is not. Review pressure moves upstream.

03 · COMPOUNDING

Learnings accrue.

Every captured pattern, every named pitfall makes the next feature faster and the next reviewer sharper.

EXPECTED OUTCOMES

What this changes, in numbers we can move.



Rework in PRs

Deviations are caught at review, not at merge.



Onboarding speed

New engineers read the spec trail and get up to speed without tribal knowledge.



Consistency

Every feature follows the same lifecycle, the same artifacts, the same review.



AI reliability

Better-shaped problems in produce better-shaped code out.

TIMING

Three trends are converging — right now.



AI adoption is accelerating.

Every team is using it. Most aren't using it well.



Complexity is rising.

Codebases grow, dependencies multiply, the cost of an inconsistent decision compounds.



Structure is the bottleneck.

Teams that build discipline around AI now will compound an advantage tools can't replicate later.

HONEST ABOUT THE RISKS

What could go wrong — and how we'll de-risk it.

RISK 01

Two external dependencies, neither owned.

OpenSpec and the Compound Engineering plugin are both third-party. Either going unmaintained breaks the workflow.

RISK 02

Plugin ecosystem is young.

Claude Code's plugin API is new. API changes can break commands without warning.

RISK 03

Abstractions hide failures.

When a command produces a bad output, root-cause attribution is opaque to a new team member.

RISK 04

No documented contract between tools.

OpenSpec and Compound Engineering share context with no formal interface — failure modes are unclear.

RECOMMENDATION · OPTION B

Keep OpenSpec for artifacts. Replace the plugin with native Claude Code commands the team owns. A few hours of work, in exchange for a workflow we control end-to-end.

We are not improving coding.
We are upgrading the engineering system
itself.



STRATEGY → SPEC → EXECUTION → LEARNING

FURTHER READING

References & further reading.

SPEC FRAMEWORKS

Spec-Kit · GitHub

github.com/github/spec-kit

OpenSpec · Fission AI

openspec.dev · github.com/Fission-AI/OpenSpec

BMAD Method

docs.bmad-method.org

Tessl

docs.tessl.io

Kiro

kiro.dev

ARTICLES & ANALYSIS

Spec-driven development unpacking 2025 · Thoughtworks

thoughtworks.com/insights/blog

Exploring Gen-AI: SDD tools · Martin Fowler

martinfowler.com/articles/exploring-gen-ai/sdd-3-tools.html

Spec-Kit vs Superpowers · dev.to

dev.to/truongpx396

What is BMAD Method? · Medium

medium.com/@visrow

LIFECYCLE & SKILLS

Compound Engineering · Every

every.to/guides/compound-engineering

Superpowers

github.com/obra/superpowers

Agent Skills · Addy Osmani

github.com/addyosmani/agent-skills

skills.sh

skills.sh

EVIDENCE & RESEARCH

What is Spec-Driven Development · Augment Code

augmentcode.com/guides/what-is-spec-driven-development

LLM vulnerability rates · Yan et al., 2025

arxiv.org/html/2602.00180v1

spec-driven.md · GitHub

github.com/github/spec-kit/blob/main/spec-driven.md